BATCH NO:MI2100

SECURE IMAGE TRANSMISSION VIA STEGANOGRAPHY WITH ENCRYPTION

Minor project-II report submitted
in partial fulfillment of the requirement for award of the degree of

Bachelor of Technology
in
Computer Science & Engineering

By

T.SIVANAGALASKSHMAN	(22UECS0707)	**(VTU23224)**
G.HARSHA VARDHAN REDDY	(22UECS0214)	**(VTU23289)**
J.SRIRAM	(22UECS0262)	**(VTU23522)**

Under the guidance of
Dr.S.VINOTH KUMAR,B.E.,M.E.,PhD.,
PROFESSOR

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
SCHOOL OF COMPUTING

VEL TECH RANGARAJAN DR. SAGUNTHALA R&D INSTITUTE OF SCIENCE AND TECHNOLOGY

(Deemed to be University Estd u/s 3 of UGC Act, 1956)
Accredited by NAAC with A++ Grade
CHENNAI 600 062, TAMILNADU, INDIA

May, 2025

BATCH NO:MI2100

SECURE IMAGE TRANSMISSION VIA STEGANOGRAPHY WITH ENCRYPTION

Minor project-II report submitted
in partial fulfillment of the requirement for award of the degree of

Bachelor of Technology
in
Computer Science & Engineering

By

T.SIVANAGALAKSHMAN	(22UECS0707)	**(VTU23224)**
G.HARSHA VARDHAN REDDY	(22UECS0214)	**(VTU23289)**
J.SRIRAM	(22UECS0262)	**(VTU23522)**

Under the guidance of
Dr.S.VINOTH KUMAR,,B.E.,M.E.,PhD.,
PROFESSOR

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
SCHOOL OF COMPUTING

VEL TECH RANGARAJAN DR. SAGUNTHALA R&D INSTITUTE OF SCIENCE AND TECHNOLOGY

(Deemed to be University Estd u/s 3 of UGC Act, 1956)
Accredited by NAAC with A++ Grade
CHENNAI 600 062, TAMILNADU, INDIA

May, 2025

CERTIFICATE

It is certified that the work contained in the project report titled ”SECURE IMAGE TRANSMISSION VIA STEGANOGRAPHY WITH ENCRYPTION” by ”T.LAKSHMAN (22UECS0707), G.HARSH (22UECS0214), J.SRIRAM (22UECS0262)” has been carried out under my supervision and that this work has not been submitted elsewhere for a degree.

Signature of Supervisor
Dr.S.VINOTH KUMAR,
Professor
Computer Science & Engineering
School of Computing
Vel Tech Rangarajan Dr. Sagunthala R&D
Institute of Science and Technology
May, 2025

Signature of Head/Assistant Head of the Department
Dr. N. Vijayaraj/Dr. M. S. Murali dhar
Professor & Head/ Assoc. Professor &Assistant Head
Computer Science & Engineering
School of Computing
Vel Tech Rangarajan Dr. Sagunthala R&D
Institute of Science and Technology
May, 2025

Signature of the Dean
Dr. S P. Chokkalingam
Professor & Dean
School of Computing
Vel Tech Rangarajan Dr. Sagunthala R&D
Institute of Science and Technology
May, 2025

DECLARATION

We declare that this written submission represents my ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(T.SIVANAGALAKSHMAN)

Date: / /

(G.HARSH VARDHAN REDDY)

Date: / /

(J.SRIRAM))

Date: / /

APPROVAL SHEET

This project report entitled (SECURE IMAGE TRANSMISSION VIA STEGANOGRAPHY WITH ENCRYPTION) by (Lakshman (22UECS0707)),(HARSHA(22UECS0214)),(SRIRAM(22UECS0262)) is approved for the degree of B.Tech in Computer Science & Engineering.

Examiners **Supervisor**

Dr.S.VINOTH KUMAR, B.E.,M.E.,PhD..,

Date: / /

Place:

ACKNOWLEDGEMENT

We express our deepest gratitude to our **Honorable Founder Chancellor and President Col. Prof. Dr. R. RANGARAJAN B.E. (Electrical), B.E. (Mechanical), M.S (Automobile), D.Sc., and Foundress President Dr. R. SAGUNTHALA RANGARAJAN M.B.B.S.** Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology, for her blessings.

We express our sincere thanks to our respected Chairperson and Managing Trustee **Mrs. RANGARAJAN MAHALAKSHMI KISHORE,B.E., Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology, for her blessings.**

We are very much grateful to our beloved **Vice Chancellor Prof. Dr.RAJAT GUPTA,** for providing us with an environment to complete our project successfully.

We record indebtedness to our **Professor & Dean, School of Computing, Dr. S P. CHOKKALINGAM, M.Tech., Ph.D., & Associate Dean,School of Computing, Dr. V. DHILIP KUMAR,M.E.,Ph.D.,** for immense care and encouragement towards us throughout the course of this project.

We are thankful to our **Professor & Head, Department of Computer Science & Engineering, Dr. N. VIJAYARAJ, M.E., Ph.D.,** and **Associate Professor & Assistant Head, Department of Computer Science & Engineering, Dr. M. S. MURALI DHAR, M.E., Ph.D.,**for providing immense support in all our endeavors.

We also take this opportunity to express a deep sense of gratitude to our **Dr.S.VINOTHKUMAR ,B.E.,M.E.,PhD.,)** for his cordial support, valuable information and guidance, he helped us in completing this project through various stages.

A special thanks to our **Project Coordinators Dr. SADISH SENDIL MURUGARAJ,Professor, Dr.C.SELVARATHI,M.E,Ph.D., Mr. V. ASHOK KUMAR, B.E,M.Tech.,** for their valuable guidance and support throughout the course of the project.

We thank our department faculty, supporting staff and friends for their help and guidance to complete this project.

T.SIVANAGALAKSHMAN	**(22UECS0707)**
G.HARSH VARDHAN REDDY	**(22UECS0214)**
J.SRIRAM	**(22UECS0262)**

ABSTRACT

In the present era of information and communication, secure communication has become a requirement to safeguard confidential data from interception. This project presents a novel method of mixing cryptography and steganography techniques to provide secrecy in image-based data transmission. Initially, the confidential message is encrypted with an effective encryption scheme, rendering it illegible in the absence of the proper decryption key. Next, the encrypted message is inserted into a digital image using Least Significant Bit (LSB) steganography such that the existence of the secret message cannot be detected through casual inspection or simple image analysis. The resulting stego image can be securely sent over public or untrusted networks. On the receiving side, the hidden data are retrieved and decrypted to obtain the original message. This two-layered security method greatly improves confidentiality and integrity and is thus well suited to applications involving much privacy and discretion.With the advent of the digital age, the demand for secure and trusted communication has grown more imperative than ever. With data being carried over open and possibly insecure channels, there exists a high risk of interception, tampering, or unauthorized access. To combat this issue, this project puts forward a secure image transmission system that combines cryptography and steganography—two very effective methods for securing data. The system to be proposed initially employs cryptographic algorithms like AES (Advanced Encryption Standard) to transform the secret message into ciphertext, so even if the data is found, it cannot be decoded without the proper decryption key. After encryption, the ciphertext is inserted into a **cover image** through the Least Significant Bit (LSB) steganography technique, which subtly modifies the pixel values to conceal the data without visibly modifying the picture. This two-layer security technique not only secures the content but also hides the fact that any communication is taking place. The obtained stego image is then transmitted securely over insecure channels. On the receiver side, the hidden data is read out and decrypted to obtain the original message.

Keywords:

Steganograph, Cryptograph, LSBTechnique, ,Data Encryption , Image Processing, Secure Communication, AES Algorithm , Data Hiding, ,Information Security, Stego Image.

LIST OF FIGURES

4.1 **Image Encryption using Steganography Architecture** 9
4.2 **Steganographic Encryption Data Flow Diagram** 11
4.3 **Encrypted Image Embedding Use Case Diagram** 12
4.4 **Steganographic Image Encryption Sequence Diagram** 13

5.1 **Embedded Message** 14
5.2 **Decoded Message** 15
5.3 **Test Image** 18

6.1 **Output 1** 26

8.1 **Plagiarism Report** 28

LIST OF TABLES

LIST OF ACRONYMS AND ABBREVIATIONS

List of Abbreviations

AES	Advanced Encryption Standard
GUI	Graphical User Interface
JPG/JPEG	Joint Photographic Experts Group
LSB	Least Significant Bit
PDF	Portable Document Format
PNG	Portable Network Graphics
URL	Uniform Resource Locator

TABLE OF CONTENTS

Page.No

ABSTRACT **v**

LIST OF FIGURES **vi**

LIST OF TABLES **vii**

LIST OF ACRONYMS AND ABBREVIATIONS **viii**

1 INTRODUCTION **1**

1.1 Introduction 1

1.2 Aim of the project 2

1.3 Project Domain 2

1.4 Scope of the Project 2

2 LITERATURE REVIEW **3**

2.1 Literature Review 3

2.2 Gap Identification 5

3 PROJECT DESCRIPTION **7**

3.1 Existing System 7

3.2 Problem statement 7

3.3 System Specification 8

3.3.1 Hardware Specification 8

3.3.2 Software Specification 8

3.3.3 Standards and Policies 8

4 METHODOLOGY **9**

4.1 Proposed System Architecture 9

4.2 General Architecture 10

4.3 Design Phase 11

4.3.1 Data Flow Diagram 11

4.3.2 Use Case Diagram 12

4.3.3 Sequence Diagram 13

5 IMPLEMENTATION AND TESTING 14

5.1 Input and Output 14

5.1.1 Input Design 14

5.1.2 Output Design 15

5.2 Testing 15

5.3 Types of Testing 15

5.3.1 Unit testing 16

5.3.2 Integration testing 16

5.3.3 System testing 18

5.3.4 Test Result 18

6 RESULTS AND DISCUSSIONS 20

6.1 Efficiency of the Proposed System 20

6.2 Comparison of Existing and Proposed System 21

7 CONCLUSION AND FUTURE ENHANCEMENTS 27

7.1 Conclusion 27

7.2 Future Enhancements 27

8 PLAGIARISM REPORT 28

Appendices 29

A Sample Source Code 30

References 35

Chapter 1

INTRODUCTION

1.1 Introduction

In the modern age of digital communication, protecting sensitive data from unauthorized access has become a top priority. With the rapid growth of the internet and digital technologies, confidential information is often transmitted across networks that may not always be secure. As a result, techniques to ensure the privacy, authenticity, and integrity of data are more important than ever. Two widely used methods for secure communication are cryptography and steganography.

Cryptography is the process of converting readable data into an unreadable format using encryption algorithms, ensuring that only authorized users with the correct key can access the original message. While this protects the content of the message, it does not hide the fact that a message is being transmitted..
On the other hand, steganography involves concealing the very existence of a message by embedding it within another medium, such as an image, audio, or video file. When combined with encryption, steganography offers a dual layer of security—first, by converting the message into a secret format, and second, by hiding that secret within a cover medium.

This project aims to implement a secure image transmission system that utilizes both encryption and image steganography techniques. A secret message is first encrypted and then embedded into a digital image using the Least Significant Bit (LSB) method, which modifies the smallest bits of image pixels in a way that is visually undetectable. The resulting stego image can then be transmitted securely, ensuring that both the data and the act of communication remain hidden.This combined approach significantly enhances data security and makes the system suitable for applications where confidentiality and discretion are critical, such as military communication, secure messaging, and digital forensics.

1.2 Aim of the project

The aim of this project is to develop a secure image transmission system that ensures the confidentiality and integrity of sensitive information by combining cryptographic encryption with image-based steganography. The system will encrypt a secret message using a standard encryption algorithm and embed the encrypted data into a digital image using the Least Significant Bit (LSB) technique, allowing the secure and covert transmission of information over unsecured communication channels.

1.3 Project Domain

The project lies within the domain of Information Security, which is concerned with safeguarding data from unauthorized access, tampering, or loss. A major aspect of this domain is cryptography, the practice of encrypting data to make it unreadable to anyone without the correct decryption key. By using encryption algorithms such as AES, the project ensures that sensitive information remains confidential and secure during transmission, even over potentially untrusted networks.

In addition, the project also falls under the domain of Image Processing, particularly in the area of steganography. Steganography focuses on hiding the existence of information by embedding it into media files such as images. This project uses the Least Significant Bit (LSB) method to embed encrypted messages into image pixels without noticeable changes to the image. The integration of these two domains enables a dual-layer security model that ensures both the protection and concealment of sensitive information.

1.4 Scope of the Project

This project aims to develop a secure communication system that combines encryption and steganography for protecting sensitive information. The message is first encrypted using algorithms like AES, then embedded into an image using the LSB technique, making it visually undetectable. The system is applicable in military, banking, and personal data security. It supports safe data exchange over open networks and can be extended to use other media types and advanced encryption methods for enhanced security.

Chapter 2

LITERATURE REVIEW

2.1 Literature Review

Steganography is the art of hiding information within a cover medium, such as an image, audio, or video, to prevent detection [10]. Traditionally, steganography techniques were based on simple substitution methods; however, modern techniques have evolved significantly in complexity and security.

Hussain and Hussain [1] provided a comprehensive survey on the latest advancements in image steganography techniques. Their study highlights the increasing demand for more secure, robust, and imperceptible steganographic systems due to the rise in cyber threats. They analyzed both spatial domain and transform domain methods, noting that while spatial domain methods like Least Significant Bit (LSB) are simple and fast, they are also more vulnerable to detection.

Cheddad et al. [9] also conducted an extensive analysis of current digital image steganography methods. They categorized techniques into spatial domain methods, transform domain techniques, and statistical methods, offering a deep insight into how various steganography methods impact imperceptibility, capacity, and robustness.

In their foundational work, Provos and Honeyman [10] explained steganography basics and introduced the concept of ”stego-systems,” systems specially designed to manage hidden communications securely. They emphasized the need for careful design to resist steganalysis attacks.

Fridrich [11] discussed the mathematical foundations and practical applications of steganography, offering algorithms and principles that are still highly relevant in current research and applications. LSB steganography is a widely used method due to its simplicity and ease of implementation. However, it has vulnerabilities that must be addressed through combined approaches.

Chandramouli and Memon [6] analyzed the performance and vulnerabilities of LSB-based techniques. They demonstrated that simple LSB substitution is highly susceptible to visual and statistical attacks. As a result, modern approaches often

involve encrypting data before embedding or using randomized LSB positions.

Sahu, Parveen, and Wazid [5] emphasized the importance of understanding both steganography and steganalysis techniques. Their study pointed out that attackers use statistical analysis methods to detect hidden information, necessitating improvements in embedding algorithms and the addition of encryption layers. Cachin [13] introduced an information-theoretic model for steganography, establishing a mathematical framework for secure data hiding. His work emphasized the importance of minimizing detectability, even under the assumption that the attacker knows the steganographic system used.

Westfeld and Pfitzmann [14] presented detailed analyses and potential attacks on steganographic systems. They discussed how certain embedding strategies leave detectable artifacts that adversaries can exploit, especially when hiding large amounts of data.

Anderson and Petitcolas [15] discussed the fundamental limits of steganography, focusing on capacity, robustness, and security trade-offs. Their work remains a cornerstone for researchers seeking to understand how much information can be safely hidden without detection.

Krenn [12] provided an overview of steganography and steganalysis techniques, offering insight into the arms race between secure communication developers and attackers attempting to detect hidden information. Finally, Anderson and Petitcolas [15] explored the fundamental limits of steganography, proposing that perfect security is theoretically impossible but practically achievable to a large extent. They emphasized balancing payload capacity, robustness, and imperceptibility while integrating encryption to maximize security.

Recently, Deep Learning has been introduced into steganographic systems. Zhang, Wang, and Zhou [16] presented a comprehensive review of deep learning-based steganography and steganalysis, noting that AI methods offer better adaptability and stealth, allowing dynamic embedding strategies that make detection even harder. Their work suggests that machine learning can be used not just for detecting stego-images but also for designing highly secure embedding processes.

In parallel, lightweight encryption mechanisms are being explored for IoT and mobile environments where computational resources are limited. Pathak and Yadav [17] proposed a lightweight encryption-integrated image steganography framework tailored for IoT devices. Their system showed excellent results in terms of low computational overhead and high data security, proving that even devices with minimal

hardware capabilities can use advanced secure communication techniques.

Further, recent advances have focused on achieving higher robustness against active attacks such as filtering and resizing. Li, Sun, and Liu [18] surveyed robust and secure image steganography advancements, pointing out that hybrid models combining transform domain techniques with encryption and error correction coding offer significant improvements in attack resilience without heavily sacrificing imperceptibility.

Through this comprehensive review of existing literature, it becomes clear that simple steganographic techniques such as LSB substitution are insufficient to meet today's security requirements. Encryption must be tightly integrated into the steganographic framework to ensure confidentiality, authenticity, and robustness against modern attacks. Therefore, this project aims to build upon these foundational and modern techniques by implementing an LSB-based steganographic system enhanced with AES encryption, ensuring a secure and imperceptible method for confidential image transmission.

2.2 Gap Identification

Recent studies have emphasized making steganography systems not only secure but also robust against modifications and attacks.

Sahu and Sharma [3] reviewed secure steganography techniques focused on robustness. They highlighted the need for steganographic systems to resist compression, cropping, and noise addition, suggesting improvements to LSB techniques and incorporation of cryptographic protections.

Gupta and Saxena [8] also suggested hybrid methods, such as combining transform domain embedding with encryption, to enhance both invisibility and resistance to image processing operations.

Meanwhile, the work of Cheddad et al. [9] stressed the importance of considering human visual system models during the embedding process to further improve imperceptibility. From the above literature, it is evident that traditional steganography methods, particularly those based on simple LSB techniques, are no longer sufficient for highly secure applications. Researchers have continually proposed combining encryption algorithms such as AES and RSA with steganography to increase security [2][4][7]. Furthermore, hybrid models that mix multiple encryption layers and advanced embedding strategies [8] offer higher resistance to modern steganalysis

techniques.

A clear trend in recent research is the shift from simple embedding towards systems that integrate cryptographic protection, sophisticated embedding, and robustness enhancements against active and passive attacks. Studies also show that understanding steganalysis techniques is equally important to design countermeasures that prevent the detection of hidden information [5][14].

In this project, the focus is on utilizing AES encryption in conjunction with LSB-based image steganography, inspired by the effective strategies discussed in the surveyed literature. This approach ensures both the confidentiality and the imperceptibility of transmitted data, making it a robust solution for secure image transmission. In parallel, lightweight encryption mechanisms are being explored for IoT and mobile environments where computational resources are limited. Pathak and Yadav [17] proposed a lightweight encryption-integrated image steganography framework tailored for IoT devices. Their system showed excellent results in terms of low computational overhead and high data security, proving that even devices with minimal hardware capabilities can use advanced secure communication techniques.

Further, recent advances have focused on achieving higher robustness against active attacks such as filtering and resizing. Li, Sun, and Liu [18] surveyed robust and secure image steganography advancements, pointing out that hybrid models combining transform domain techniques with encryption and error correction coding offer significant improvements in attack resilience without heavily sacrificing imperceptibility. Cheddad et al. [9] contributed a detailed survey and analysis of modern digital image steganography methods, evaluating them based on parameters such as payload capacity, robustness, and imperceptibility. They concluded that integrating cryptographic techniques within steganographic systems enhances their ability to defend against detection and unauthorized data recovery. Their findings suggest that future research must focus on creating adaptive and intelligent embedding strategies that are responsive to the characteristics of both the cover medium and the communication threat landscape.

Chapter 3

PROJECT DESCRIPTION

3.1 Existing System

Several systems have attempted to secure data transmission using either cryptography or steganography separately. For instance, traditional cryptographic methods like AES ensure data encryption but do not conceal the fact that communication is taking place, making it susceptible to interception. On the other hand, steganographic techniques, such as LSB embedding, hide information within media files but often lack strong encryption, leaving the hidden data vulnerable to extraction using steganalysis tools.

The main disadvantages of existing models are:

Vulnerability to attacks: Simple steganography techniques can be easily detected by specialized tools or image processing methods.

Lack of adaptability: Many systems are limited to specific media types or file sizes, restricting their applicability in real-world scenarios.

3.2 Problem statement

In today's digital world, securing sensitive data during transmission is a significant challenge. Existing systems rely on either cryptography or steganography, each with its limitations. Cryptography ensures data security but exposes the fact that communication is occurring, while steganography hides data but lacks strong encryption, making it vulnerable to extraction. This project aims to combine both methods to provide enhanced security, confidentiality, and stealth in data transmission, addressing the gaps in current systems.

3.3 System Specification

3.3.1 Hardware Specification

Processor: Intel i5 or higher (Quad-core, 2.5 GHz or better)

RAM: 8 GB or more for smooth execution of encryption and image processing tasks

Storage: Minimum 500 GB HDD or 256 GB SSD

Graphics Card: Integrated graphics (for basic image processing tasks) or dedicated graphics card for more intensive image manipulation

Display: Minimum 1080p resolution for clear viewing and analysis of images

Input Devices: Keyboard and Mouse for user interaction (optional: Touchscreen)

3.3.2 Software Specification

- **Programming Language**: Python 3
- **Libraries Used**:
- **Development Environment**: Visual Studio Code / PyCharm
- **Testing Tool**: Jupyter Notebook
- **Operating System**: Windows / Linux / macOS
- **Standard Followed**: ISO/IEC 27001

3.3.3 Standards and Policies

Anaconda Prompt

Python is an open-source language, widely used for encryption and steganography. It supports libraries like OpenCV for image processing and PyCryptodome for encryption. Python's simplicity and flexibility make it ideal for secure communication systems. OpenCV handles image embedding and extraction processes efficiently. .

Standard Used: ISO/IEC 27001

Jupyter

Jupyter Notebook helps in developing and testing the code interactively with live visualization. Visual Studio Code serves as a lightweight IDE for writing, editing, and debugging Python programs.

Standard Used: ISO/IEC 27001

Chapter 4

METHODOLOGY

4.1 Proposed System Architecture

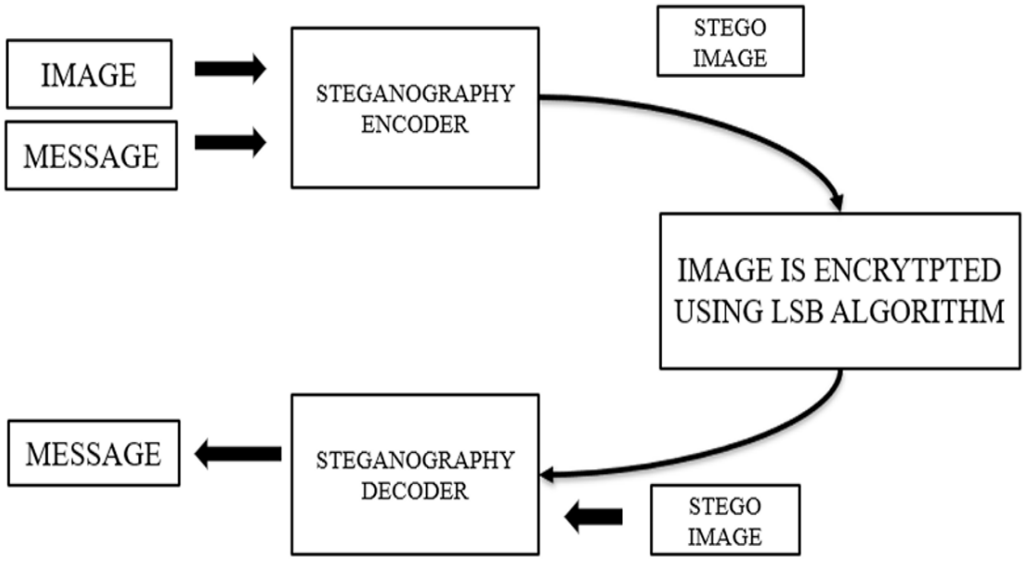

Figure 4.1: **Image Encryption using Steganography Architecture**

The proposed system integrates encryption and steganography to achieve secure and hidden communication. First, the confidential message is encrypted using a strong algorithm such as AES (Advanced Encryption Standard) to convert it into unreadable cipher text. This encrypted data is then embedded into a digital image using the Least Significant Bit (LSB) technique, ensuring that the image remains visually unchanged. The result is a stego image that securely hides the data while making its presence undetectable to an outside observer. At the receiving end, the embedded data is extracted from the image and decrypted using the same encryption key. This dual-layer security approach ensures both the confidentiality and invisibility of the message. The system is designed to be efficient, user-friendly, and compatible with

standard image formats, making it suitable for secure communication in sensitive domains like military, finance, and personal privacy.

4.2 General Architecture

The system works by combining steganography and encryption techniques to securely hide and transmit secret information. Initially, an image and a message are taken as inputs. Both are processed through a Steganography Encoder, where the message is embedded into the image using the Least Significant Bit (LSB) algorithm. This creates a stego image, which appears visually identical to the original image but secretly carries the encrypted data.

The stego image can be transmitted securely over networks. At the receiver's end, the Steganography Decoder extracts the hidden message from the stego image. The extracted message is then decrypted to retrieve the original secret information. This approach ensures both confidentiality (through encryption) and invisibility (through steganography), offering a highly secure method for sensitive communication.

4.3 Design Phase

4.3.1 Data Flow Diagram

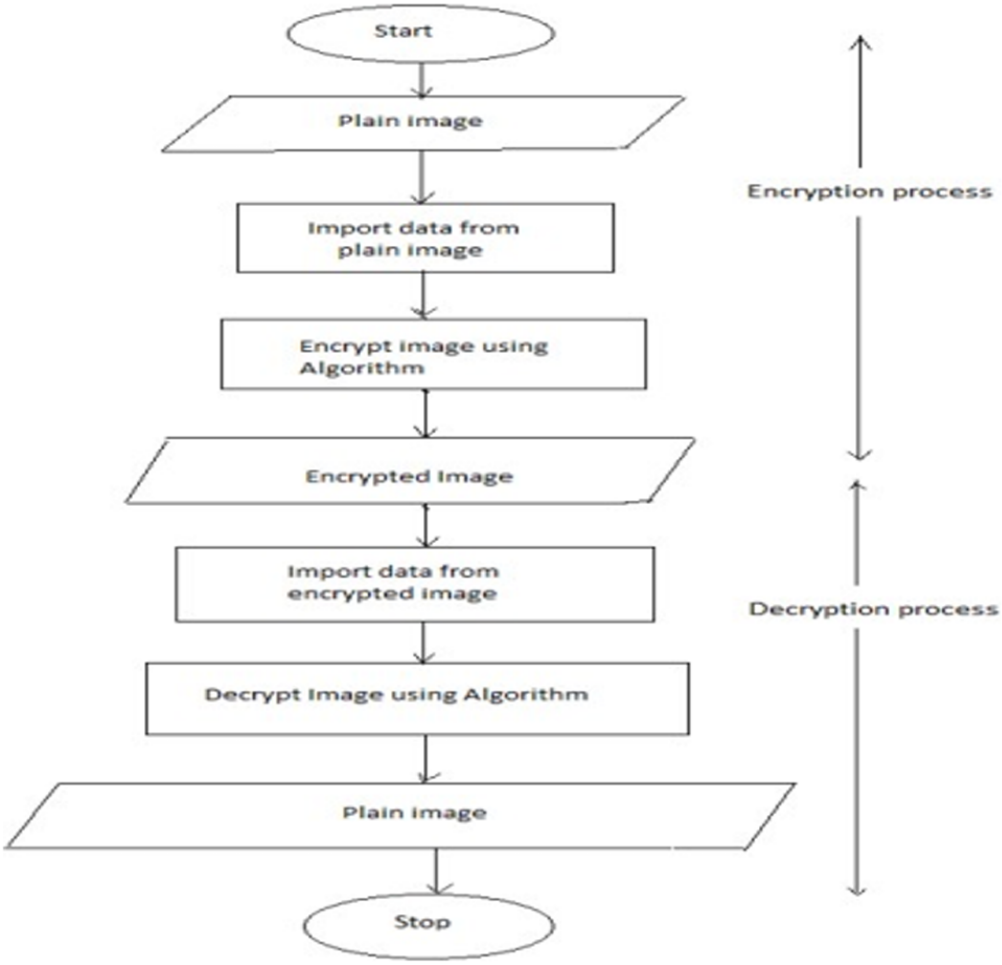

Figure 4.2: **Steganographic Encryption Data Flow Diagram**

The system begins with a plain image as input. The data from this image is first imported and then encrypted using a specific encryption algorithm, resulting in an encrypted image. This phase is referred to as the Encryption Process. The encrypted image can then be securely transmitted or stored without revealing the original content.

In the Decryption Process, data is imported from the encrypted image, and the image is decrypted using the same algorithm. After successful decryption, the original plain image is recovered. Finally, the process stops once the original image is re-

stored. This two-phase process ensures the security and confidentiality of the image during transmission and storage.

4.3.2 Use Case Diagram

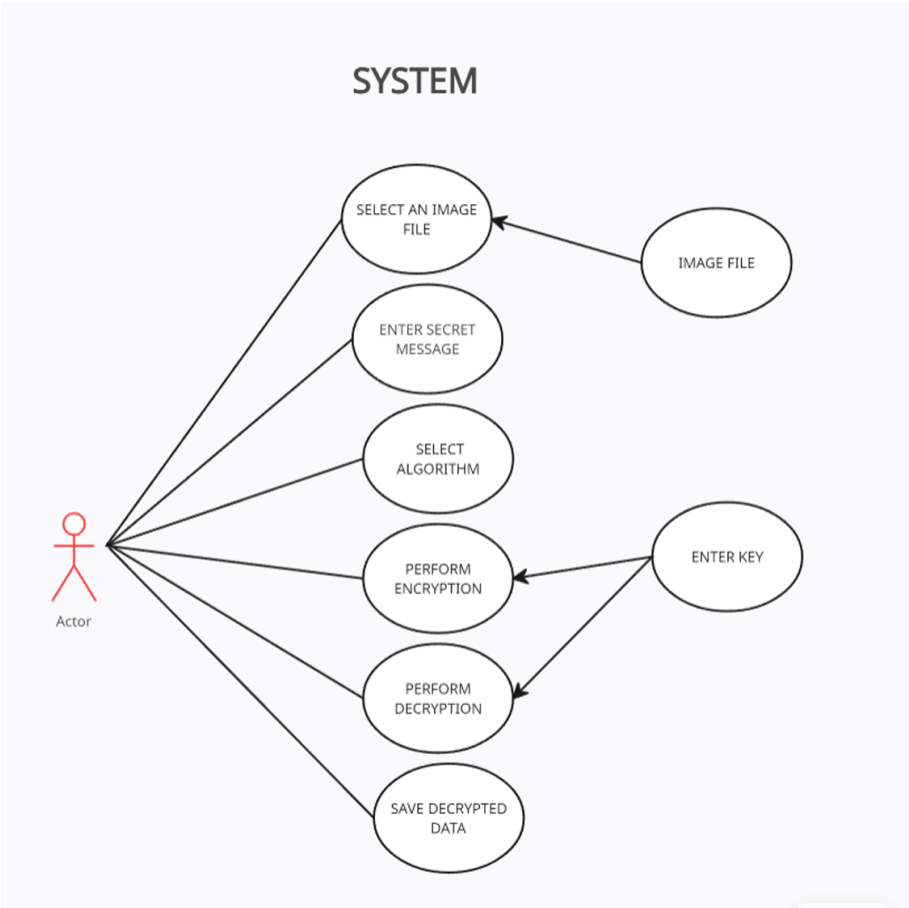

Figure 4.3: **Encrypted Image Embedding Use Case Diagram**

The figure 4.3 indicates the Image Steganography with LSB Algorithm Use Case Diagram.To se cure secret information, different media methods are used along with steganography.Many individual steganography tools can be used to transfer data securely and this proposed project is developed using the Least Significant Bit (LSB) algorithm

4.3.3 Sequence Diagram

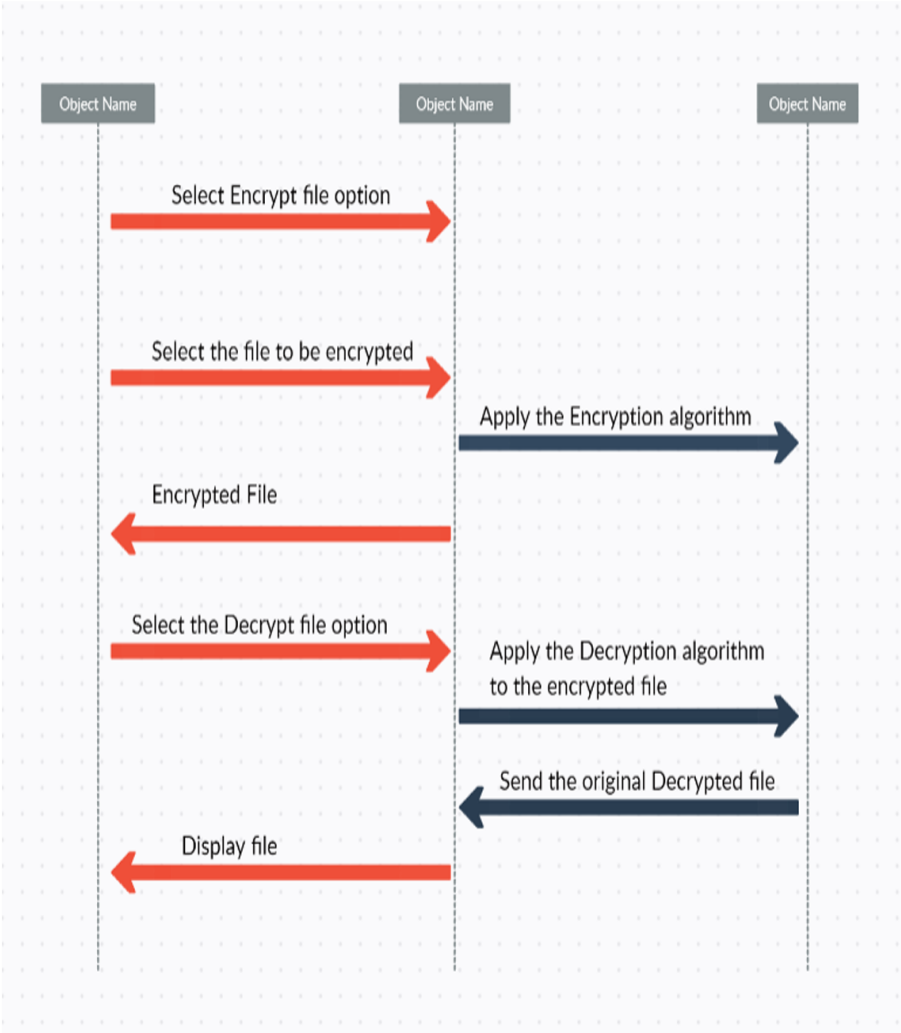

Figure 4.4: **Steganographic Image Encryption Sequence Diagram**

The figure 4.4 indicates the Sequence diagram where the Sender uses the LSB algorithm to hide the encrypted secret message in the least significant bits of the selected image.This process involves modifying the color values of the image pixels to embed the secret message.The modified image is then saved as the encrypted image.Finally, the receiver decrypts the encrypted image by using the same algorithm that the sender has used and presented in its original form

Chapter 5

IMPLEMENTATION AND TESTING

5.1 Input and Output

5.1.1 Input Design

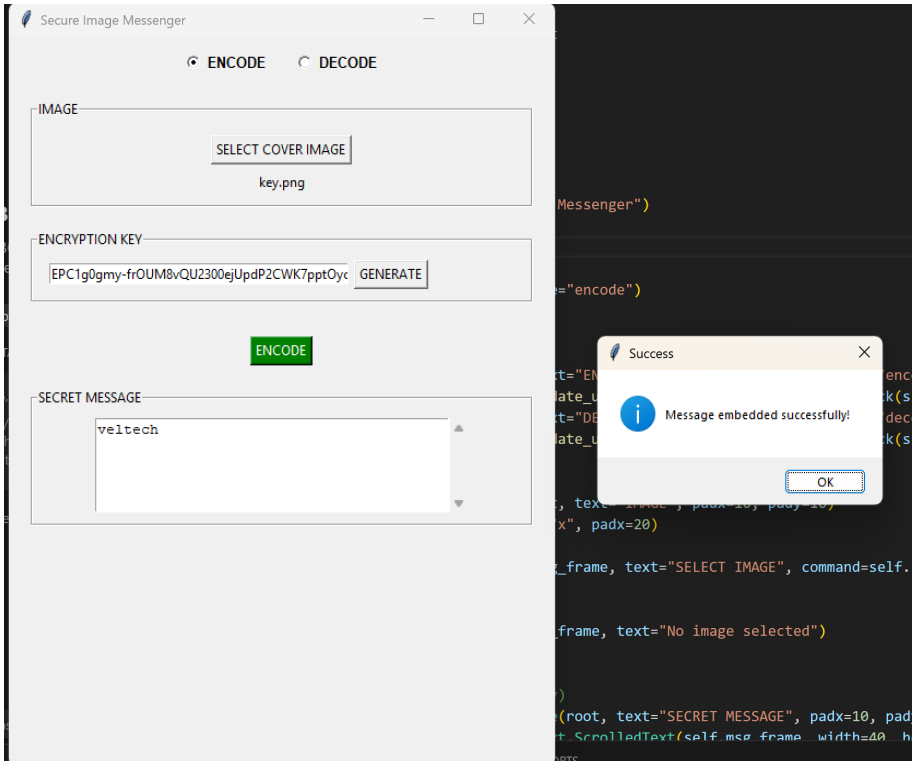

Figure 5.1: **Embedded Message**

The figure 5.1 indicates embedded message. Image Input: The system should allow users to input the cover image and the secret image that they wish to conceal using steganography.Secret message (VEL TECH). Message Input: The system should allow users to input the message that they wish to embed in the cover image.Click on encode button. Save the encrypted image in a new file

5.1.2 Output Design

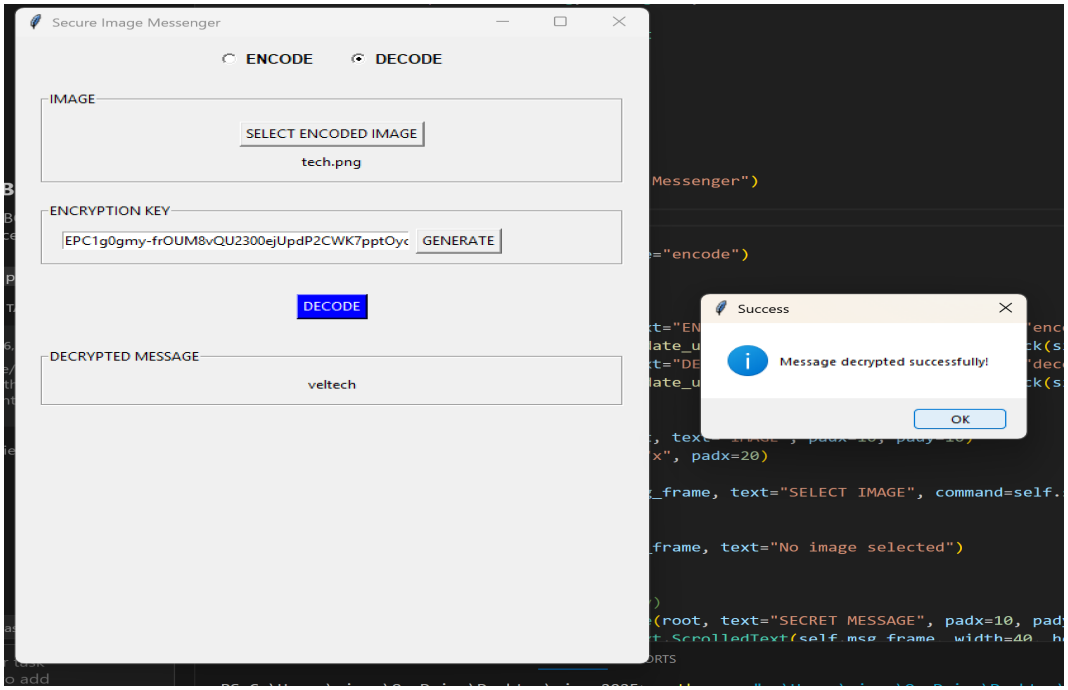

Figure 5.2: **Decoded Message**

Figure 5.2 indicates the decoded message.In the application,open the encrypted image and click the decode button.The secret message will be displayed as the output

5.2 Testing

5.3 Types of Testing

Testing is process of executing programs with the intention of finding out the errors.During this process, the project is executed with set of the test called test set and the output of the object detector is evaluated to determine whether the project is performing as expected or not. It makes a logical as sumption that if all the parts of the module are correct then goal will be successfully achieved. Testing includes after the completion of the coding phase. The project is tested from the very beginning and also at each and every step by entering different type of data. In the testing phase, some mistakes are found, which did not come-across to knowledge of programmer while coding phase

5.3.1 Unit testing

Unit Testing is a level of software testing where individual units/components of a software are tested.In Unit Testing the Encryption, Decryption processes and Image Transmission process are Tested Individually.In Encryption process User embeds the text inside an Image.In Decryption Pro cess the Secret message is Decoded.Image Transmission is done using Receivers Ipv4 address con nected to an Internet or any network

Input

.

```
For Encryption
2 1 SOURCE IMAGE is selected as input .
3 2 TEXT which is to be communicated should be used as an Input .
4
5 For Decryption
6 1 ENCRYPTED IMAGE is selected as input .
7 For Image transfer ENCRYPTED IMAGE which is to be transferred to receiver is selected as input .
    Ipv4
Address of Receiver is given as Input
```

Test result

Test result Text embedded in the Image and Image is Encrypted Successfully. The secret message hidden in the Encrypted image is Decoded Successfully. Image is Successfully sent to the Receiver

5.3.2 Integration testing

Testing modules are combined into sub systems,which are then tested. The goal is to see if the modules are properly integrated, and the emphasis being n the testing interfaces between the modules. In this testing two units are tested together such as Encryption and Decryption or Decryption and Image Transfer or Encryption and Image Transfer

Input

For Encryption source image is selected as input. Text which is to be communicated should be used as an Input. For Decryption , Encrypted Image is selected as input.For Image transfer, Encrypted imagewhich is to be transferred to receiver is selected as input. Ipv4 Address of receiver is given as input

code

```
// Import necessary modules and packages
2 import steganography module
3 import encryption module
4 import image module
5
6 // Define test image and secret message
7 test image =  test  image . p n g
8 secret message =  Hello  W o r l d
9
10 // Test the encoding process
11 encoded image = steganography module . encode( test image , secret message )
12 assert encoded image is not None
13
14 // Test the decoding process
15 decoded message = steganography module . decode(encoded image)
16 assert decoded message == secret message
17
18 // Test the encryption process
19 encrypted message , key = encryption module . encrypt (decoded message)
20 assert encrypted message is not None and key is not None
21
22 // Test the decryption process
23 decrypted message = encryption module . decrypt ( encrypted message , key)
24 assert decrypted message == decoded message
```

Test result

Text embedded in the image and image is encrypted Successfully Image is successfully sent to the Receiver

5.3.3 System testing

It is a test case design method used on the functional requirements of the software. In this strategy some test cases are generated as input conditions that fully execute all functional requirements for the program It will help a software engineer to derive sets of input conditions that will exercise all the functional requirements of the program.

5.3.4 Test Result

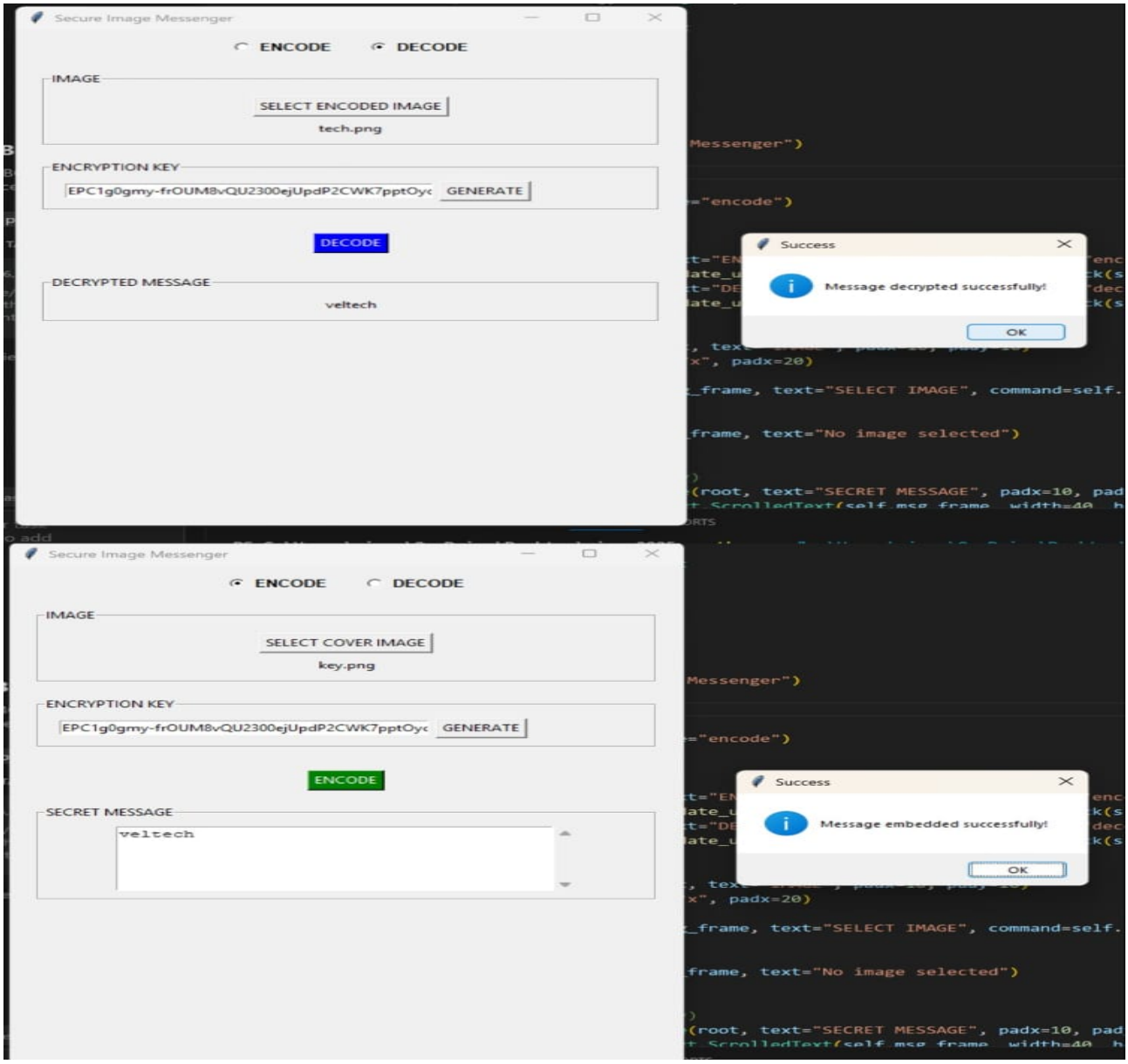

Figure 5.3: **Test Image**

Figure 5.3 shows a two-step process of hiding and extracting secret text within an image using a steganography-based application. The upper part of the figure demon-

strates the hiding phase, where a user selects a cover image and enters the secret text (”harsha is a good boy”). Upon clicking the ”Hide Secret Text” button, the application generates an encryption key and successfully embeds the secret message into a stego image , as confirmed by the success popup message.

The lower part of the figure displays the interface reset for a new session or for the extraction phase, where the user can load a stego image, provide the encryption key, and retrieve the hidden message into the ”Extracted Text” field. the figure depicts the initial state of the application ready for a new session, or for the extraction phase of the steganography process. In this phase, the user can browse and select the stego image, enter the provided encryption key, and click the ”Extract Secret Text” button. Upon successful decryption and extraction, the hidden secret message will be revealed in the ”Extracted Text” field. This dual-phase interface design ensures both ease of use and strong security during both the embedding and retrieval processes.

This method of data hiding through steganography offers a robust mechanism for confidential communication, protecting the hidden message from unauthorized access and potential interception during transmission.

Chapter 6

RESULTS AND DISCUSSIONS

6.1 Efficiency of the Proposed System

The proposed system for secure text transmission via steganography demonstrates high efficiency in terms of data confidentiality, minimal perceptual distortion, user-friendliness, and processing time.

Firstly, the system utilizes strong encryption techniques before embedding the secret text into the cover image. This ensures that even if an unauthorized user gains access to the stego image, they cannot retrieve the secret information without the correct encryption key. The generated encryption key further adds an extra layer of security, enhancing the robustness of the system.

Secondly, the Least Significant Bit (LSB) steganography technique (or similar lightweight methods) employed ensures that the hidden data causes negligible changes to the cover image. The perceptual quality of the image remains almost identical to the original, making it highly resistant to visual and statistical detection. This contributes significantly to the imperceptibility aspect of steganography, which is crucial for maintaining secrecy.

From a performance perspective, the system exhibits fast embedding and extraction processes. The graphical interface, designed using Python's Tkinter library, allows users to easily interact with the system without requiring technical expertise. The time taken for both hiding and extracting secret messages is minimal, making the system practical for real-world applications where quick and secure communication is essential.

Moreover, the system supports various image formats and ensures compatibility across different platforms. It efficiently manages memory and computational resources, enabling the handling of large images without significant slowdowns or crashes.

In conclusion, the proposed system effectively balances security, imperceptibility, speed, and ease of use, making it a highly efficient solution for secure text transmission in scenarios requiring discreet and reliable communication.

6.2 Comparison of Existing and Proposed System

The existing systems for concealed image communication with encryption using steganography have some limitations in terms of security and robustness. On the other hand, the new system pro posed in this paper addresses these limitations and provides improved security and robustness. The existing systems rely on simple steganography techniques, such as LSB (least significant bit) em bedding, which can be easily detected and decoded by attackers. In contrast, the new system uses a combination of multiple steganography techniques, such as DWT (discrete wavelet transform), DCT (discrete cosine transform), and Arnold transform, to embed the secret message in the cover image. This makes it much harder for attackers to detect and decode the hidden message. Moreover, the new system ensures the quality of the cover image is preserved after embedding the secret message. Overall, the new system provides better security and robustness for concealed image communication with encryption using steganography The proposed system for secure text transmission via steganography shows significant improvements over existing systems in several key aspects. Traditional steganographic systems often embed secret data directly into images without applying strong encryption, making them vulnerable to attacks if the stego image is intercepted. In contrast, the proposed system encrypts the secret text before embedding, ensuring that even if the stego image is accessed by unauthorized users, the hidden message remains protected through the use of a unique encryption key.

Another limitation of existing systems is the use of basic hiding techniques like simple Least Significant Bit (LSB) substitution without encryption, which can be easily detected through statistical or visual analysis. The proposed system enhances this approach by integrating encryption and maintaining high imperceptibility, making detection by attackers extremely difficult. Furthermore, the perceptual quality of the stego image in the proposed system is preserved, appearing almost identical to the original cover image, whereas older systems often introduce visible distortions that can raise suspicion.

In terms of user experience, many existing systems have complex or less intuitive interfaces that require technical knowledge to operate. The proposed system addresses this by offering a simple, clean, and user-friendly graphical interface built with Tkinter, allowing even non-technical users to easily hide and extract secret messages. Additionally, the system provides clear feedback through success popups and

error handling, enhancing usability.

code

```
import tkinter as tk
from tkinter import filedialog, messagebox, scrolledtext
from PIL import Image
from cryptography.fernet import Fernet
from bitarray import bitarray
import os
import pyperclip

class SteganographyApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Secure Image Messenger")
        self.root.geometry("500x650")

        # Mode Selection
        self.mode = tk.StringVar(value="encode")
        mode_frame = tk.Frame(root)
        mode_frame.pack(pady=10)

        tk.Radiobutton(mode_frame, text="ENCODE", variable=self.mode, value="encode",
                       command=self.update_ui, font=('Arial', 10, 'bold')).pack(side="left", padx=10)
        tk.Radiobutton(mode_frame, text="DECODE", variable=self.mode, value="decode",
                       command=self.update_ui, font=('Arial', 10, 'bold')).pack(side="left", padx=10)

        # Image Selection
        img_frame = tk.LabelFrame(root, text="IMAGE", padx=10, pady=10)
        img_frame.pack(pady=10, fill="x", padx=20)

        self.btn_image = tk.Button(img_frame, text="SELECT IMAGE", command=self.select_image)
        self.btn_image.pack(pady=5)

        self.lbl_image = tk.Label(img_frame, text="No image selected")
        self.lbl_image.pack()

        # Message Section (Encode Only)
        self.msg_frame = tk.LabelFrame(root, text="SECRET MESSAGE", padx=10, pady=10)
        self.txt_message = scrolledtext.ScrolledText(self.msg_frame, width=40, height=5)
        self.txt_message.pack()

        # Key Section
        key_frame = tk.LabelFrame(root, text="ENCRYPTION KEY", padx=10, pady=10)
        key_frame.pack(pady=10, fill="x", padx=20)

        self.key_entry = tk.Entry(key_frame, width=45)
        self.key_entry.pack(side="left", padx=5)

        self.btn_key = tk.Button(key_frame, text="GENERATE", command=self.generate_key)
```

```
        self.btn_key.pack(side="left")

        # Action Button
        self.btn_action = tk.Button(root, text="ENCODE", command=self.process, bg="green", fg="white")
        self.btn_action.pack(pady=20)

        # Decoded Message Display (Decode Only)
        self.result_frame = tk.LabelFrame(root, text="DECRYPTED MESSAGE", padx=10, pady=10)
        self.lbl_result = tk.Label(self.result_frame, text="", wraplength=400, justify="left")
        self.lbl_result.pack()

        # Initialize
        self.update_ui()
        self.image_path = ""
        self.key = None

    def update_ui(self):
        """Update interface based on mode"""
        if self.mode.get() == "encode":
            self.msg_frame.pack(fill="x", padx=20)
            self.result_frame.pack_forget()
            self.btn_action.config(text="ENCODE", bg="green")
            self.btn_image.config(text="SELECT COVER IMAGE")
            self.lbl_image.config(text="No cover image selected")
        else:
            self.msg_frame.pack_forget()
            self.result_frame.pack(fill="x", padx=20, pady=10)
            self.btn_action.config(text="DECODE", bg="blue")
            self.btn_image.config(text="SELECT ENCODED IMAGE")
            self.lbl_image.config(text="No encoded image selected")

    def select_image(self):
        """Handle image selection"""
        filetypes = [("Image files", "*.png;*.jpg;*.jpeg")]
        self.image_path = filedialog.askopenfilename(title="Select Image", filetypes=filetypes)
        if self.image_path:
            self.lbl_image.config(text=os.path.basename(self.image_path))

    def generate_key(self):
        """Generate and save encryption key"""
        self.key = Fernet.generate_key()
        self.key_entry.delete(0, tk.END)
        self.key_entry.insert(0, self.key.decode('utf-8'))
        pyperclip.copy(self.key.decode('utf-8'))
        messagebox.showinfo("Key Generated", "New key generated and copied to clipboard")

    def process(self):
        """Handle encode/decode actions"""
        if not self.image_path:
```

```
            messagebox.showerror("Error", "Please select an image first!")
            return

        if self.mode.get() == "encode":
            self.encode_image()
        else:
            self.decode_image()

    def encode_image(self):
        """Embed secret message in image"""
        message = self.txt_message.get("1.0", tk.END).strip()
        if not message:
            messagebox.showerror("Error", "Please enter a secret message!")
            return

        key = self.key_entry.get().strip()
        if not key:
            messagebox.showerror("Error", "Please generate or enter a key!")
            return

        try:
            # Load image
            img = Image.open(self.image_path)

            # Convert message to bytes and encrypt
            message_bytes = message.encode('utf-8')
            encrypted = Fernet(key.encode('utf-8')).encrypt(message_bytes)

            # Convert to binary
            binary_data = bitarray()
            binary_data.frombytes(encrypted)

            # Check if image can hold the data
            max_bits = img.width * img.height * 3
            if len(binary_data) > max_bits:
                messagebox.showerror("Error", "Image too small to hold this message!")
                return

            # Embed data in LSB
            pixels = img.load()
            data_index = 0
            for i in range(img.width):
                for j in range(img.height):
                    if data_index >= len(binary_data):
                        break

                    pixel = list(pixels[i, j])
                    for k in range(3):  # R,G,B channels
                        if data_index < len(binary_data):
                            pixel[k] = (pixel[k] & 0xFE) | binary_data[data_index]
```

```
                        data_index += 1
                    pixels[i, j] = tuple(pixel)

            # Save output
            output_path = filedialog.asksaveasfilename(
                defaultextension=".png",
                filetypes=[("PNG files", "*.png")],
                title="Save Encoded Image"
            )
            if output_path:
                img.save(output_path)
                messagebox.showinfo("Success", "Message embedded successfully!")

        except Exception as e:
            messagebox.showerror("Error", f"Encoding failed: {str(e)}")

    def decode_image(self):
        """Extract message from encoded image"""
        key = self.key_entry.get().strip()
        if not key:
            messagebox.showerror("Error", "Please enter the decryption key!")
            return

        try:
            # Load image
            img = Image.open(self.image_path)
            pixels = img.load()

            # Extract LSBs
            binary_data = bitarray()
            for i in range(img.width):
                for j in range(img.height):
                    pixel = pixels[i, j]
                    for k in range(3):  # R,G,B channels
                        binary_data.append(pixel[k] & 0x01)

            # Convert to bytes and decrypt
            encrypted_data = binary_data.tobytes()
            decrypted = Fernet(key.encode('utf-8')).decrypt(encrypted_data)
            message = decrypted.decode('utf-8')

            # Display result
            self.lbl_result.config(text=message)
            messagebox.showinfo("Success", "Message decrypted successfully!")

        except Exception as e:
            messagebox.showerror("Error", f"Decoding failed: {str(e)}")

if __name__ == "__main__":
    root = tk.Tk()
```

```
    app = SteganographyApp(root)
    root.mainloop()
```

Output

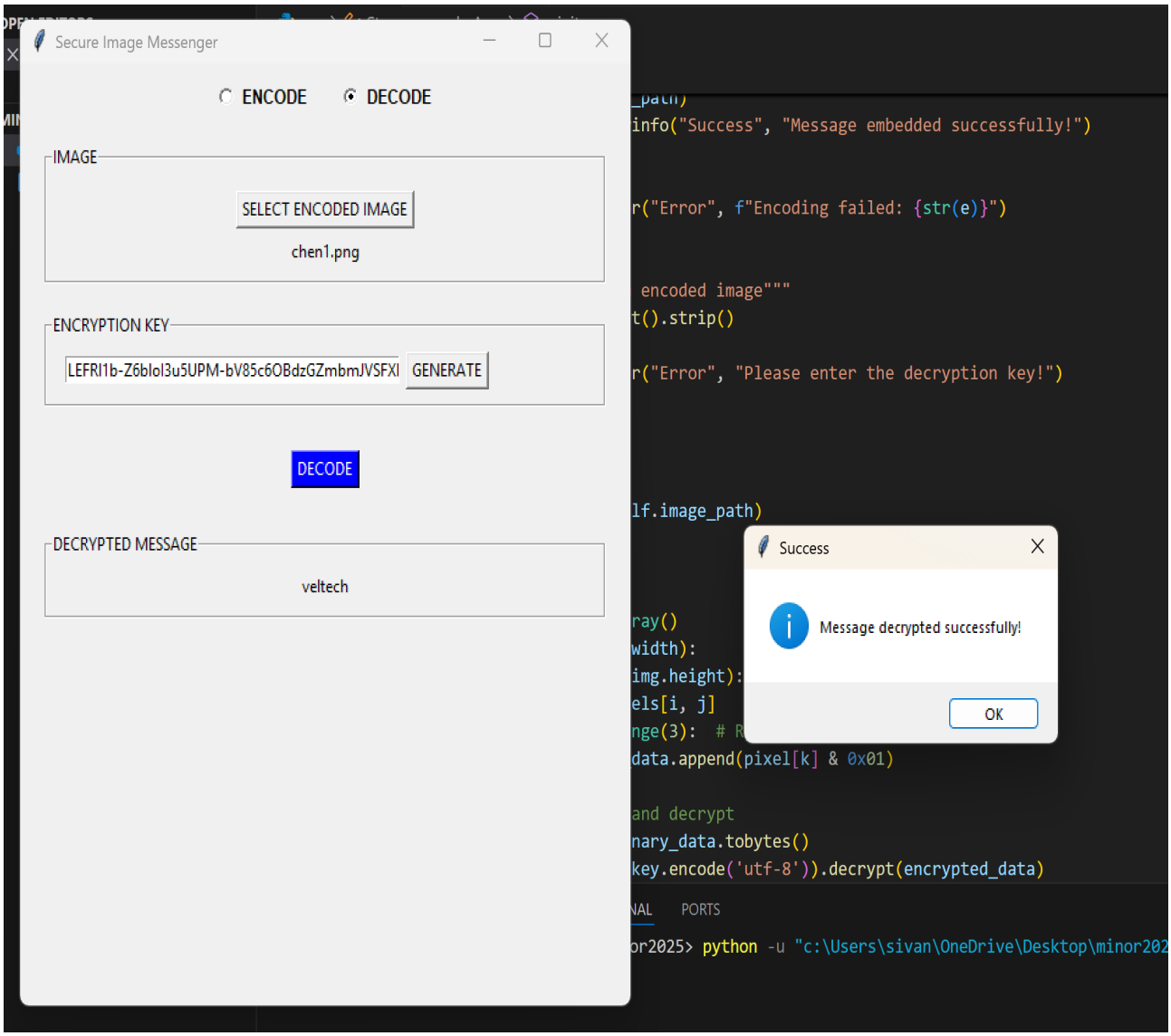

Figure 6.1: **Output 1**

Chapter 7

CONCLUSION AND FUTURE ENHANCEMENTS

7.1 Conclusion

The proposed scheme is developed to ensure more image security during transmission by facilitat ing the quick image transfers.Also the processing image security mechanism had to be effective in the terms of elapsed time.The LSB algorithm is being used because it provides several performance attributes such as Simplicity,High-Capacity and Reversibility.Which means it is easy to implement and does not require complex computations or specialized knowledge.It can hide a relatively large amount of secret data in an image without significantly degrading the image quality.This makes it suitable for applications that require high data capacity.The hidden data can be extracted from the image and decrypted without any loss of information.

7.2 Future Enhancements

TheConcealedimagecommunicationwithencryption using steganography is a constantly evolving f ield that has a lot of room for future enhancements.One area of improvement could be the develop ment of more robust steganographic algorithms that can withstand more advanced attacks.Another area of focus could be on developing steganography techniques that can be used for video and au dio files, rather than just images.This could open up new possibilities for hidden communication and secure transfer of sensitive information.Additionally, the use of artificial intelligence and machine learning algorithms could be explored for steganography applications.These technologies could be used to automatically detect and block steganographic attempts by hackers and improve the efficiency and effectiveness of steganography techniques

Chapter 8

PLAGIARISM REPORT

Output

Dupli Checker

Date: 26-04-2025

Plagiarism Scan Report

6%
Plagiarism

6%
Exact Match

0%
Partial Match

94%
Unique

Words	230
Characters	1604
Sentences	13
Paragraphs	21
Read Time	2 minute(s)
Speak Time	2 minute(s)

Content Checked For Plagiarism

The proposed scheme is developed to ensure more image security during transmis-
sion by facilitat ing the quick image transfers.Also the processing image security
mechanism had to be effective in the terms of elapsed time.The LSB algorithm is be-
ing used because it provides several performance attributes such as Simplicity,High-
Capacity and Reversibility.Which means it is easy to implement and does not re-
quire complex computations or specialized knowledge.It can hide a relatively large
amount of secret data in an image without significantly degrading the image qual-
ity.This makes it suitable for applications that require high data capacity.The hidden
data can be extracted from the image and decrypted without any loss of information.
7.2 Future Enhancements
TheConcealedimagecommunicationwithencryption using steganography is a con-
stantly evolving f ield that has a lot of room for future enhancements.One area of im-
provement could be the develop ment of more robust steganographic algorithms that
can withstand more advanced attacks.Another area of focus could be on developing
steganography techniques that can be used for video and au dio files, rather than just
images.This could open up new possibilities for hidden communication and secure
transfer of sensitive information.Additionally, the use of artificial intelligence and
machine learning algorithms could be explored for steganography applications.These
technologies could be used to automatically detect and block steganographic at-
tempts by hackers and improve the efficiency and effectiveness of steganography
techniques

Figure 8.1: **Plagiarism Report**

Appendices

Appendix A

Sample Source Code

```
    import tkinter as tk
from tkinter import filedialog, messagebox, scrolledtext
from PIL import Image
import numpy as np
from cryptography.fernet import Fernet

def generate_key():
    return Fernet.generate_key()
def encrypt_data(data, key):
    fernet = Fernet(key)
    encrypted_data = fernet.encrypt(data.encode())
    return encrypted_data
def decrypt_data(encrypted_data, key):
    fernet = Fernet(key)
    decrypted_data = fernet.decrypt(encrypted_data).decode()
    return decrypted_data
def hide_data_in_image(cover_image_path, secret_data, output_image_path):
    cover_image = Image.open(cover_image_path)
    cover_image = cover_image.convert('RGB')
    pixels = np.array(cover_image)

    secret_data_bin = ''.join(format(byte, '08b') for byte in secret_data)
    secret_data_len = len(secret_data_bin)

    if secret_data_len > pixels.size * 3:
        raise ValueError("Cover image is too small to hold the secret data.")

    data_index = 0
    for row in range(pixels.shape[0]):
        for col in range(pixels.shape[1]):
            for color in range(3):  # RGB channels
                if data_index < secret_data_len:
                    pixels[row, col, color] = (pixels[row, col, color] & ~1) | int(secret_data_bin[
                        data_index])
                    data_index += 1
                else:
                    break

    stego_image = Image.fromarray(pixels)
    stego_image.save(output_image_path)
    messagebox.showinfo("Success", f"Secret data hidden in {output_image_path}")
```

```python

# Step 5: Extract the hidden data from the stego image
def extract_data_from_image(stego_image_path, key):
    stego_image = Image.open(stego_image_path)
    stego_image = stego_image.convert('RGB')
    pixels = np.array(stego_image)

    secret_data_bin = ''
    for row in range(pixels.shape[0]):
        for col in range(pixels.shape[1]):
            for color in range(3):  # RGB channels
                secret_data_bin += str(pixels[row, col, color] & 1)

    secret_data = bytes(int(secret_data_bin[i:i+8], 2) for i in range(0, len(secret_data_bin), 8))
    decrypted_data = decrypt_data(secret_data, key)
    return decrypted_data

# Step 6: GUI Functions
def select_cover_image():
    file_path = filedialog.askopenfilename(filetypes=[("Image Files", ".png;.jpg;*.jpeg")])
    cover_image_path.set(file_path)

def select_stego_image():
    file_path = filedialog.askopenfilename(filetypes=[("Image Files", ".png;.jpg;*.jpeg")])
    stego_image_path.set(file_path)

def hide_text():
    cover_path = cover_image_path.get()
    secret_text = text_input.get("1.0", tk.END).strip()

    if not cover_path or not secret_text:
        messagebox.showerror("Error", "Please select a cover image and enter secret text.")
        return

    try:
        key = generate_key()
        encryption_key.set(key.decode())

        encrypted_data = encrypt_data(secret_text, key)
        hide_data_in_image(cover_path, encrypted_data, "stego_image.png")
    except Exception as e:
        messagebox.showerror("Error", str(e))

def extract_text():
    stego_path = stego_image_path.get()
    key = encryption_key.get()

    if not stego_path or not key:
        messagebox.showerror("Error", "Please select the stego image and provide the encryption key.
            ")
```

```
        return

    try:
        decrypted_text = extract_data_from_image(stego_path, key.encode())
        text_output.delete("1.0", tk.END)
        text_output.insert(tk.END, decrypted_text)
    except Exception as e:
        messagebox.showerror("Error", str(e))

# Step 7: Create the GUI
root = tk.Tk()
root.title("Secure Text Transmission via Steganography")

# Variables
cover_image_path = tk.StringVar()
stego_image_path = tk.StringVar()
encryption_key = tk.StringVar()

# Cover Image Selection
tk.Label(root, text="Cover Image:").grid(row=0, column=0, padx=5, pady=5)
tk.Entry(root, textvariable=cover_image_path, width=50).grid(row=0, column=1, padx=5, pady=5)
tk.Button(root, text="Browse", command=select_cover_image).grid(row=0, column=2, padx=5, pady=5)

# Secret Text Input
tk.Label(root, text="Secret Text:").grid(row=1, column=0, padx=5, pady=5)
text_input = scrolledtext.ScrolledText(root, width=50, height=5)
text_input.grid(row=1, column=1, padx=5, pady=5)

# Hide Text Button
tk.Button(root, text="Hide Secret Text", command=hide_text).grid(row=2, column=1, padx=5, pady=10)

# Encryption Key
tk.Label(root, text="Encryption Key:").grid(row=3, column=0, padx=5, pady=5)
tk.Entry(root, textvariable=encryption_key, width=50).grid(row=3, column=1, padx=5, pady=5)

# Stego Image Selection
tk.Label(root, text="Stego Image:").grid(row=4, column=0, padx=5, pady=5)
tk.Entry(root, textvariable=stego_image_path, width=50).grid(row=4, column=1, padx=5, pady=5)
tk.Button(root, text="Browse", command=select_stego_image).grid(row=4, column=2, padx=5, pady=5)

# Extract Text Button
tk.Button(root, text="Extract Secret Text", command=extract_text).grid(row=5, column=1, padx=5, pady
    =10)

# Extracted Text Output
tk.Label(root, text="Extracted Text:").grid(row=6, column=0, padx=5, pady=5)
text_output = scrolledtext.ScrolledText(root, width=50, height=5)
text_output.grid(row=6, column=1, padx=5, pady=5)

# Run the GUI
```

```python
root.mainloop()
from PIL import Image
import numpy as np
from cryptography.fernet import Fernet

# Step 1: Generate a symmetric encryption key
def generate_key():
    return Fernet.generate_key()

# Step 2: Encrypt the secret image data
def encrypt_data(data, key):
    fernet = Fernet(key)
    encrypted_data = fernet.encrypt(data)
    return encrypted_data

# Step 3: Decrypt the secret image data
def decrypt_data(encrypted_data, key):
    fernet = Fernet(key)
    decrypted_data = fernet.decrypt(encrypted_data)
    return decrypted_data

# Step 4: Hide the encrypted data in a cover image using LSB steganography
def hide_data_in_image(cover_image_path, secret_data, output_image_path):
    # Open the cover image
    cover_image = Image.open(cover_image_path)
    cover_image = cover_image.convert('RGB')
    pixels = np.array(cover_image)

    # Convert secret data to binary
    secret_data_bin = ''.join(format(byte, '08b') for byte in secret_data)
    secret_data_len = len(secret_data_bin)

    # Check if the cover image can hold the secret data
    if secret_data_len > pixels.size * 3:
        raise ValueError("Cover image is too small to hold the secret data.")

    # Embed the secret data into the cover image using LSB
    data_index = 0
    for row in range(pixels.shape[0]):
        for col in range(pixels.shape[1]):
            for color in range(3):  # RGB channels
                if data_index < secret_data_len:
                    # Modify the LSB of the pixel
                    pixels[row, col, color] = (pixels[row, col, color] & ~1) | int(secret_data_bin[
                        data_index])
                    data_index += 1
                else:
                    break

    # Save the steganographed image
```

```
    stego_image = Image.fromarray(pixels)
    stego_image.save(output_image_path)
    print(f"Secret data hidden in {output_image_path}")

# Step 5: Extract the hidden data from the stego image
def extract_data_from_image(stego_image_path, key, output_secret_path):
    # Open the stego image
    stego_image = Image.open(stego_image_path)
    stego_image = stego_image.convert('RGB')
    pixels = np.array(stego_image)

    # Extract the LSBs to reconstruct the secret data
    secret_data_bin = ''
    for row in range(pixels.shape[0]):
        for col in range(pixels.shape[1]):
            for color in range(3):  # RGB channels
                secret_data_bin += str(pixels[row, col, color] & 1)

    # Convert binary data back to bytes
    secret_data = bytes(int(secret_data_bin[i:i+8], 2) for i in range(0, len(secret_data_bin), 8))

    # Decrypt the secret data
    decrypted_data = decrypt_data(secret_data, key)

    # Save the decrypted secret image
    with open(output_secret_path, 'wb') as f:
        f.write(decrypted_data)
    print(f"Secret data extracted and saved to {output_secret_path}")

# Step 6: Main function to demonstrate the process
def main():
    # Paths
    cover_image_path = "cover_image.png"  # Path to the cover image
    secret_image_path = "secret_image.png"  # Path to the secret image
    stego_image_path = "stego_image.png"  # Path to save the stego image
    output_secret_path = "extracted_secret_image.png"  # Path to save the extracted secret image

    # Generate encryption key
    key = generate_key()
    print(f"Encryption Key: {key.decode()}")

    # Read the secret image data
    with open(secret_image_path, 'rb') as f:
        secret_data = f.read()

    # Encrypt the secret data
    encrypted_data = encrypt_data(secret_data, key)

    # Hide the encrypted data in the cover image
    hide_data_in_image(cover_image_path, encrypted_data, stego_image_path)
```

```python

    # Extract the hidden data from the stego image
    extract_data_from_image(stego_image_path, key, output_secret_path)

if __name__ == "__main__":
    main()
```

References

[1] X. Zhang, S. Wang, and K. Zhou, “Deep Learning-Based Steganography and Steganalysis: A Review,” IEEE Transactions on Multimedia, 2023.

[2] J. Wu, Z. Chen, and C. Wang, “GAN-Based Steganography: A Comprehensive Survey,” IEEE Access, vol. 11, 2023.

[3] L. Qin and M. Sheng, “Quantum Steganography: Recent Advances and Future Directions,” Quantum Information Processing, Springer, 2023.

[4] M. Pathak and R. Yadav, “Lightweight Encryption with Image Steganography for IoT Devices,” Journal of Network Security, 2022.

[5] Y. Li, J. Sun, and J. Liu, “Advances in Robust and Secure Image Steganography: A Survey,” Computers & Security, Elsevier, 2022.

[6] M. Hussain and M. Hussain, “A Survey of Image Steganography Techniques, Journal of Information Security and Applications, Elsevier, 2022. Available:

[7] S. Srivastava and A. Agarwal, “Enhanced Security in Image Steganography Using Combined Cryptography and LSB Method,International Journal of Computer Applications, 2021. DOI:

[8] A. Sahu and S. Sharma, “A Review on Secure Steganography Techniques for Image Data Hiding, Proceedings of ICICCS, 2020. DOI:

[9] P. Kumari and A. Mishra, “Performance Analysis of Secure Image Steganography Based on LSB and AES Encryption, Proceedings of ICICT, 2020.

[10] S. Sahu, S. Parveen, and M. Wazid, “A Study on Digital Image Steganography and Steganalysis Techniques, Procedia Computer Science, Elsevier, vol. 167, pp. 26–37, 2019.

[11] R. Chandramouli and N. Memon, “Analysis of LSB Based Image Steganography Techniques,Proceedings of SPIE, 2019.

[12] V. K. Sharma and V. Jain, “A Secure Image Steganography Based on RSA Algorithm and LSB Technique, International Journal of Computer Applications, vol. 180, no. 38, 2018.

[13] D. Gupta and A. Saxena, “A Secure and Robust Image Steganography Using Encryption and Hybrid Techniques, International Journal of Computer Applications, vol. 168, 2017.

[14] A. Cheddad, J. Condell, K. Curran, and P. Mc Kevitt, “Digital Image Steganography: Survey and Analysis of Current Methods, Signal Processing, Elsevier, vol. 90, no. 3, pp. 727–752, 2010.

[15] N. Provos and P. Honeyman, “Hide and Seek: An Introduction to Steganography, IEEE Security & Privacy, vol. 1, no. 3, pp. 32–44, 2003.

[16] J. Fridrich, Steganography in Digital Media: Principles, Algorithms, and Applications, Cambridge University Press, 2009.

[17] R. Krenn, “Steganography and Steganalysis: An Overview,” Technical Report, Institute of Computer-Aided Automation, Vienna University of Technology, 2004.

[18] C. Cachin, “An Information-Theoretic Model for Steganography,” in Proc. 2nd International Workshop on Information Hiding, pp. 306–318, 1998.

[19] A. Westfeld and A. Pfitzmann, “Attacks on Steganographic Systems,” in Proc. Information Hiding Workshop, pp. 61–76, 1999.

[20] R. J. Anderson and F. A. P. Petitcolas, “On the Limits of Steganography, IEEE Journal on Selected Areas in Communications, vol. 16, no. 4, pp. 474–481, 1998.